# MNEMORAG: Exploiting Inter-Request Correlations for High-Performance RAG Serving

Xin Yan[1], Hang Huang[1], Zhuo Huang[1], Song Wu[1], Hai Jin[1]
[1]National Engineering Research Center for Big Data Technology and System,
Services Computing Technology and System Lab, Cluster and Grid Computing Lab,
School of Computer Science and Technology, Huazhong University of Science and Technology,
Wuhan, China
{lemonyx, hanghuang, huangzhuo, wusong, hjin}@hust.edu.cn

***Abstract*—Retrieval-Augmented Generation (RAG) grounds large language models (LLMs) in external knowledge, but its practical deployment is hindered by substantial serving overheads from expensive vector retrieval, long-context prefill computation, and the strict serialization of retrieval and generation. Existing approaches largely optimize these stages in isolation, overlooking exploitable correlations across requests. We present MNEMORAG, a correlation-aware RAG serving system that systematically exploits inter-request correlations to jointly optimize retrieval and generation. MNEMORAG introduces *similarity-aware retrieval*, which leverages historical query–result associations to dynamically narrow the search space or directly reuse retrieval results, thereby eliminating redundant vector searches. It further employs *hotness-driven prefix caching*, which exploits skewed document popularity and prompt-order flexibility to maximize KV-cache reuse and reduce prefill computation. Finally, MNEMORAG adopts *asynchronous RAG scheduling* to decouple retrieval and generation across requests, enabling cross-stage overlap and improving resource utilization. Extensive evaluations across diverse RAG workloads and real-world datasets show that MNEMORAG consistently outperforms state-of-the-art systems, delivering $1.38\times$–$3.77\times$ end-to-end speedups while preserving retrieval accuracy and generation quality.**

***Index Terms*—component, formatting, style, styling, insert.**

## I. INTRODUCTION

*Large Language Models* (LLMs) have become a foundational technology in modern natural language processing, demonstrating strong capabilities in reasoning, tool use, and natural language understanding and generation [1]. Built on transformer architectures and trained on large-scale datasets, LLMs encode rich linguistic and factual knowledge within their parameters [2], enabling accurate and context-aware inference for a wide range of tasks [3], [4]. However, reliance solely on *parametric knowledge* can become a limitation in knowledge-intensive applications (e.g., medical diagnostics, financial fraud detection, and legal research [5]–[7]) that demand precise factual grounding. In such scenarios, LLMs may produce hallucinations [8], where outputs are factually incorrect or fabricated, thereby compromising reliability.

To address this challenge, *Retrieval-Augmented Generation* (RAG) [9] has emerged as a practical approach that augments LLMs with external, domain-specific knowledge to improve factual accuracy [10], [11]. As shown in Figure 1, RAG typically follows a two-stage serving pipeline in which

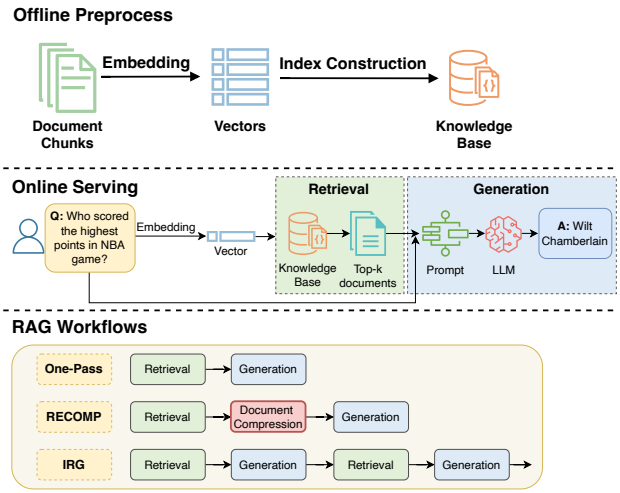



Fig. 1. RAG system overview.

**retrieval precedes generation**. During retrieval, similarity search is performed over a preprocessed knowledge base, such as a vector database, to identify the top-$k$ documents most relevant to the user query. These documents are then combined with the query to form an *augmented prompt* for LLM inference [12]. By grounding generation in retrieved evidence, RAG mitigates hallucinations without costly task-specific fine-tuning, making it a broadly applicable paradigm for knowledge-intensive applications [13]–[15].

However, despite its effectiveness, RAG fundamentally reshapes the serving pipeline by introducing a latency-critical retrieval path and substantially expanding the computational footprint of generation. Consequently, RAG serving often incurs markedly higher end-to-end latency and lower throughput than retrieval-free LLM serving, posing a major barrier to deployment at scale [16], [17]. During retrieval, similarity search algorithms such as *Approximate Nearest Neighbor Search* (ANNS) are executed over large-scale vector indices (e.g., IVF [18]); such operations are often CPU-intensive and can contribute considerable latency [19]–[21]. The retrieved documents further expand the input context, increasing the computation and memory-access overhead of the LLM prefill

stage[1] [22], [23]. More importantly, the strict inter-stage dependency places retrieval and generation on a sequential critical path, limiting opportunities for execution overlap and amplifying both average and tail latency [15], [16], [24]–[26]. Collectively, these overheads create a fundamental trade-off between knowledge augmentation and serving efficiency, necessitating cross-stage optimization for RAG serving.

Recent studies have explored stage-wise and system-level optimizations across the retrieval–generation pipeline. In the retrieval stage, HaS [27] validates speculative retrieval from restricted scopes through query homology, while Proximity [28] reuses cached document results for similar queries. QVCache [29] caches retrieved vectors and validates ANN cache hits using adaptive regional thresholds. In the generation stage, CacheBlend [30] and Cache-Craft [31] reuse KV caches with selective recomputation to reduce redundant prefill computation. RAGCache [22] caches the KV states of retrieved documents, while ContextPilot [32] improves prefix-cache reuse by reorganizing overlapping contexts. Beyond stage-specific optimizations, RAGO [33] optimizes task placement, batching, and resource allocation; HedraRAG [34] coordinates retrieval and generation across heterogeneous resources; and FlashRAG [17] provides a modular framework for constructing and evaluating complete RAG pipelines. However, these approaches exploit cross-request correlations only partially, including shared retrieval results, reusable generation states, and overlapping contexts. This lack of holistic coordination limits resource efficiency and leaves substantial performance gains unexplored in multi-request serving.

In this paper, we present an empirical study of inter-request correlations using real-world workloads and identify three key opportunities for optimizing RAG serving. First, semantically similar requests often retrieve highly overlapping document sets and even identical top-$k$ results, enabling retrieval result reuse to eliminate redundant similarity searches. Second, document accesses exhibit strong skewness, where a small subset of documents dominates retrieval frequency, while generation quality remains largely insensitive to document ordering. This property enables hotness-aware KV cache management to improve prefix cache hit rates and reduce redundant prefill computation. Third, although retrieval and generation are sequentially dependent within individual requests, such dependency does not exist across requests, allowing decoupled scheduling to reduce pipeline serialization and improve overall serving efficiency.

Based on these observations, we propose MNEMORAG, a high-performance RAG system that exploits inter-request correlations to jointly optimize retrieval and generation. (1) To eliminate redundant retrieval computation, MNEMORAG introduces *similarity-aware retrieval* (§IV-B), which leverages historical query–retrieval correlations to dynamically adjust the search strategy. By maintaining a lightweight *Query Base* of previous queries and retrieval results, MNEMORAG either reuses existing results for highly similar queries or narrows the search space for related ones. (2) To accelerate generation, MNEMORAG presents *hotness-driven prefix caching* (§IV-C), which exploits document popularity skewness and prompt-order flexibility to maximize KV cache reuse. It prioritizes frequently accessed documents to improve prefix sharing and employs hotness-aware cache management to retain valuable KV states. (3) To overcome pipeline serialization, MNEMORAG introduces *asynchronous RAG scheduling* (§IV-D), which decouples retrieval and generation across requests, enabling their overlap and improving resource utilization.

We implement MNEMORAG with approximately 3K lines of code atop Faiss [35] and vLLM [36]. Extensive evaluations across diverse RAG workloads and real-world datasets show that MNEMORAG consistently outperforms state-of-the-art baselines, achieving $1.38\times$–$3.77\times$ speedups while preserving retrieval accuracy and generation quality. These results establish inter-request correlation as an effective systems-level optimization for scalable RAG serving.

The key contributions of this paper are as follows:

- We characterize the performance of RAG serving under realistic workloads and uncover three optimization opportunities arising from inter-request correlations: retrieval-result overlap, skewed document popularity, and cross-request pipeline independence.
- We design and implement MNEMORAG, a correlation-aware high-performance RAG serving system that jointly optimizes retrieval and generation through similarity-aware retrieval, hotness-driven prefix caching, and asynchronous RAG scheduling.
- We evaluate MNEMORAG on realistic RAG workloads, delivering substantial end-to-end speedups without compromising retrieval accuracy or generation quality.

## II. BACKGROUND

This section characterizes the architecture and performance bottlenecks of RAG serving. Section II-A presents the end-to-end serving workflow, while Sections II-B and II-C identify vector search and prefill computation as the dominant latency sources in retrieval and generation, respectively. These observations motivate exploiting inter-request correlations for end-to-end serving optimization.

### A. RAG System and Serving Workflow

A standard *Retrieval-Augmented Generation (RAG)* system consists of tightly integrated retrieval and generation modules that collectively enable knowledge-enhanced generation [9], [37]. The retrieval module maintains a vectorized knowledge base in which documents are encoded into high-dimensional embeddings and partitioned into semantically coherent chunks, and is integrated with high-performance vector search libraries such as Faiss [35]. It supports efficient indexing, scalable similarity search, and retrieval of semantically relevant documents in response to arbitrary queries [38]–[40]. The generation module is built on a large language model (LLM) and deployed through inference engines such as vLLM [36]

[1]The prefill stage of LLM inference processes all input tokens to construct the key-value (KV) cache before autoregressive decoding begins.

and SGLang [41], producing output conditioned on both the user query and the retrieved documents [14]. Modern serving engines incorporate advanced optimizations including prefix caching and sequence batching to achieve low-latency, high-throughput inference, enabling seamless coordination between retrieval and generation for interactive, multi-turn applications [23], [42], [43].

Retrieval and generation form the core execution pipeline of RAG serving. As illustrated in Figure 1, conventional RAG systems typically adopt a one-pass retrieve–then–generate workflow, in which each query triggers a single retrieval operation whose results are directly incorporated into the generation prompt. Although simple and efficient, this workflow is highly dependent on retrieval quality and is susceptible to the *lost-in-the-middle* effect when the retrieved context is noisy or excessively long [44]. To reduce long-context processing overhead while preserving salient evidence, retrieve–compress–generate workflows, exemplified by RECOMP-style methods [45]–[48], insert an intermediate compression or refinement stage before generation. In contrast, iterative retrieval–generation (IRG) workflows [49]–[52] repeatedly interleave retrieval and generation, using intermediate outputs, such as refined queries or reasoning traces, to guide subsequent retrieval and progressively improve the supporting context. Collectively, these workflow variants enhance the efficiency and robustness of RAG under long or noisy contexts, while introducing increasingly heterogeneous stage compositions and inter-stage dependencies that demand workflow-aware serving optimization.

### B. Retrieval via Vector Search

Retrieval is typically performed via vector search, which identifies query-relevant embeddings from a large-scale vectorized knowledge base. The input query is first encoded into a high-dimensional vector based on the embedding model (e.g., 768 dimensions for BERT [38] and 1024 for E5-Large [39]). The system then applies the nearest neighbor search (NNS) over the embedding space to retrieve the top-$k$ vectors based on similarity metrics such as cosine similarity, inner product, or Euclidean distance [53]. Retrieval quality is commonly measured by Recall@$k$, which measures the proportion of relevant documents retrieved within the top-$k$ results. The retrieved vectors are subsequently mapped to their corresponding documents and supplied to the generation stage as supporting context.

As knowledge bases continue to grow, exact nearest-neighbor search becomes prohibitively expensive because it requires exhaustive distance evaluations over millions or billions of vectors [12]. Practical RAG systems therefore rely on approximate nearest-neighbor search (ANNS) techniques [54]–[57], such as IVF [18] and HNSW [54], which substantially accelerate retrieval at the cost of a small, controllable loss in recall. Among them, IVF is widely adopted because of its scalability and efficiency. IVF partitions the embedding space via coarse quantization, grouping vectors around learned centroids during index construction [58], and structures query processing as a two-stage pipeline. A coarse search first selects

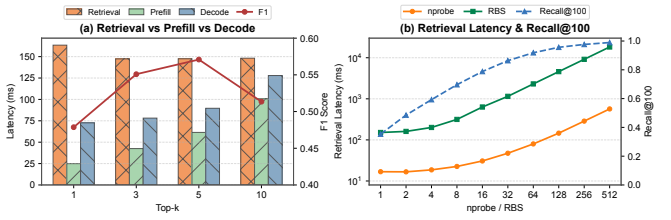



Fig. 2. Retrieval latency characterization in RAG serving. (a) Stage-wise latency across varying top-$k$. (b) Impact of IVF parameters on retrieval latency and quality.

the top-$nprobe$ nearest clusters, followed by a fine-grained search over vectors within these clusters. Practical systems further batch multiple queries during retrieval, with the *retrieval batch size* (RBS) controlling the number of queries processed concurrently. Thus, $nprobe$ governs the trade-off between search accuracy and computational cost, whereas RBS determines the degree of parallelism and achievable retrieval throughput [56].

We conduct an empirical study on the NQ [59] dataset to characterize the latency breakdown of RAG serving. We set the request rate to 1 req/s and vary top-$k$ under a representative IVF configuration ($nprobe = 128$, RBS $= 256$) to characterize the retrieval latency due to vector search. As shown in Figure 2(a), retrieval latency is largely insensitive to top-$k$, indicating that the dominant cost comes from index traversal rather than the number of retrieved results. Consequently, retrieval contributes substantially to end-to-end latency and *Time To First Token* (TTFT)[2]. Meanwhile, F1 increases from 0.479 at top-$k = 1$ to 0.571 at top-$k = 5$, but drops to 0.513 at top-$k = 10$, indicating that excessive retrieval increases generation cost while introducing distracting context. We further fix top-$k$ to 3 for latency measurements and independently vary $nprobe$ and RBS, while evaluating Recall@100 across different $nprobe$ settings. Figure 2(b) shows that increasing $nprobe$ improves Recall@100 by exploring more clusters but introduces additional search latency, revealing an inherent quality–efficiency trade-off. In contrast, increasing RBS improves retrieval throughput by amortizing computation and memory access costs, while increasing individual query latency due to batching delays. Overall, these results establish retrieval as a major performance bottleneck and highlight the need for coordinated optimization of top-$k$, $nprobe$, and RBS to balance retrieval quality, latency, and throughput.

### C. Generation with Retrieved Documents

A fundamental distinction between RAG serving and conventional LLM serving lies in their prompt construction paradigms. Instead of directly processing user queries, RAG systems construct an *augmented prompt* by integrating retrieved documents with system instructions and the original query. This retrieval-augmented context introduces substantial variability in prompt length and composition, making generation efficiency highly sensitive to retrieval context. Since LLM serving processes the entire input context during prefill

[2]TTFT captures the latency from request arrival to the first output token, including retrieval and prefill overheads before decoding.

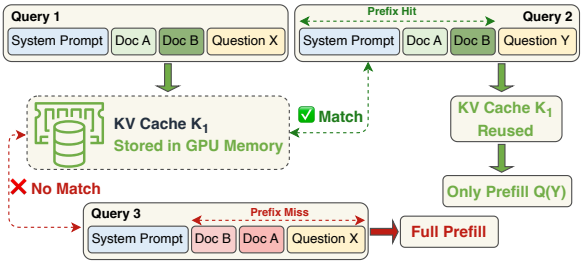



Fig. 3. Prefix KV cache reuse under exact matching. Matching prefixes enable partial prefill via KV cache reuse, while mismatches incur full prefill.

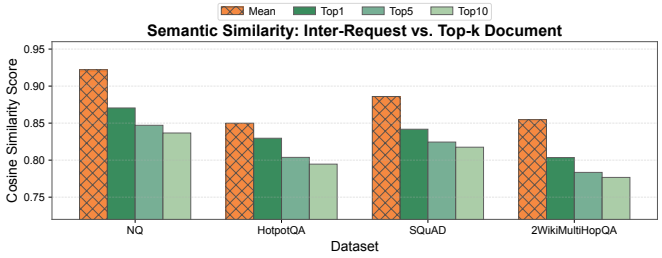



Fig. 4. Mean inter-request semantic similarity compared to top-$k$ retrieval similarity.

before autoregressive decoding, longer RAG prompts can significantly increase prefill computation [60]. To mitigate such overhead, modern LLM serving systems employ KV-cache management and reuse techniques.

Among these techniques, prefix caching extends KV-cache reuse across requests by reusing previously computed KV states when requests share identical prefixes. This approach avoids redundant prefill computation and is particularly promising for RAG workloads, where requests often share system prompts and overlapping retrieved content [22], [61]–[63]. However, its effectiveness fundamentally depends on strict and exact token-level prefix matching across requests. As KV states are deterministically derived from the entire prefix sequence, including token identities and positional encodings [42], even minor prompt variations can cause prefix mismatches and limit cache reuse.

This limitation becomes more pronounced in RAG systems, where prompt construction is tightly coupled with retrieval outputs. As illustrated in Figure 3, KV cache reuse is only possible when requests retrieve identical documents in the same order. Any variation in document content or ordering breaks prefix matching and invalidates cached KV states. Existing serving engines [17], [36], [41] provide limited support for such retrieval-induced prompt variability, resulting in low prefix KV cache hit rates for RAG workloads. As shown in Figure 2(a), this limitation directly translates into increased prefill latency. Our measurements (Figure 14) further demonstrate that increasing top-$k$ sharply reduces prefix matching rates (e.g., from 20.35% at top-1 to 2.84% at top-10).

## III. Motivation

In this section, we characterize inter-request behaviors in RAG serving and uncover three optimization opportunities: semantic correlation among requests, skewed document popularity, and cross-request execution independence. These observations uncover untapped optimization opportunities and drive the design of MnemoRAG.

### A. Inter-request Semantic Similarity

Existing RAG systems perform retrieval independently for each request, treating queries in isolation [16], [17]. The associated similarity computation over large-scale vector spaces remains expensive, despite advances in vector search techniques. When requests exhibit semantic correlations, they often access overlapping regions of the embedding space and retrieve similar document sets. However, existing systems overlook such cross-request redundancy, leading to repeated similarity searches and unnecessary retrieval overhead.

To characterize this opportunity, we analyze several real-world RAG workloads and uncover substantial inter-request semantic similarity arising from recurring information-seeking patterns. As illustrated in Figure 4, queries exhibit strong semantic alignment with their retrieved top-$k$ documents. More importantly, most queries have highly related counterparts within the workload, with average pairwise similarity exceeding 0.85, comparable to the similarity between queries and their retrieved documents. These results demonstrate that semantic locality is an intrinsic property of RAG workloads rather than an incidental artifact. Such locality implies that queries need not be retrieved in isolation: semantically related historical requests provide useful priors for locating relevant documents, reducing redundant exploration of the vector index. This motivates *similarity-aware retrieval* in MnemoRAG, which translates inter-request semantic locality into lower retrieval cost.

### B. Document Access Skewness

The effectiveness of prefix caching is a key factor determining RAG serving efficiency, as it directly reduces redundant prefill computation during generation. However, achieving high cache utilization in RAG is challenging because cached KV states can only be reused when future requests exhibit sufficient prefix overlap with previous prompts. Unlike conventional LLM workloads, RAG prompts are dynamically constructed from retrieved documents, whose access patterns exhibit strong document-level locality. Existing serving engines primarily rely on token-level prefix matching and generic cache management policies, such as least-recently-used (LRU) or first-come-first-served (FCFS) [36], [41], limiting their ability to exploit document-level reuse opportunities in RAG workloads. As a result, frequently accessed documents may fail to contribute to effective prefix reuse, leaving substantial optimization opportunities unexplored.

We analyze several real-world RAG workloads and observe pronounced skewness in document access patterns. As shown in Figure 5, document accesses are highly concentrated on a small subset of frequently retrieved documents, indicating substantial document-level locality across requests. However, existing prefix caching mechanisms fail to fully exploit this

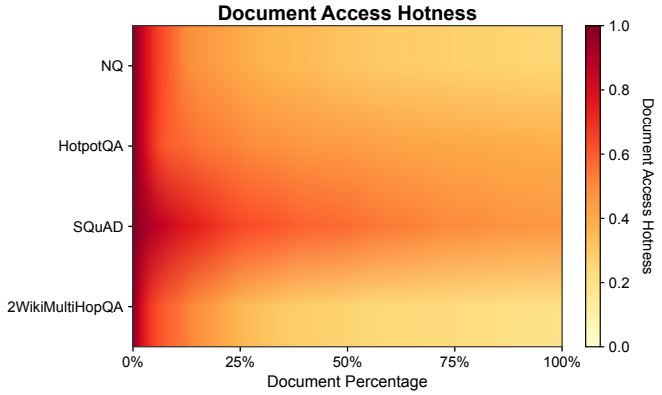



Fig. 5. Document access distribution.

locality due to their rigid token-level matching requirement: even minor variations in prompt construction can invalidate cached KV states [64], [65]. In particular, retrieved documents are typically concatenated according to retrieval order, which may vary across requests and cause prompts with highly overlapping content to diverge [9].

This observation reveals a key opportunity: retrieved documents form an unordered semantic context whose effectiveness depends on information coverage rather than positional arrangement. Our empirical analysis further shows that reordering documents within the same retrieved set using a lightweight *hint* preserves generation accuracy. This insight motivates *hotness-driven prefix caching* in MNEMORAG, which leverages document access skewness to improve prefix KV cache reuse and reduce redundant prefill computation.

### C. Independent Inter-request Execution

Contemporary RAG systems typically adopt a request-centric execution model, where retrieval and generation are strictly serialized within each request [16], [17]. Existing serving frameworks such as LangChain [16] and FlashRAG [17] often propagate this intra-request dependency to batch-level scheduling, deferring retrieval for subsequent requests until the current batch completes generation. This cross-request ordering is not semantically required, as preparing one request is independent of generating another.

HedraRAG [34] exploits this independence through fine-grained task orchestration, enabling retrieval and generation from different requests to execute concurrently. Yet overlap alone does not remove retrieval from the critical path: any retrieval tail that exceeds the generation window remains exposed, while speculative execution may incur correction or regeneration. Effective pipelining therefore depends on whether subsequent request preparation can be fully contained within the available overlap window.

This issue is particularly pronounced in MNEMORAG, where request preparation extends beyond conventional CPU-side vector search [35] to include GPU-side query processing and the restoration of reusable prefix KV blocks from CPU memory [36]. A request becomes generation-ready only after these retrieval and data-movement stages complete. Serializing the entire preparation path behind preceding generations therefore places both costs on the end-to-end critical path. Their independence from ongoing generation motivates the resource-aware *asynchronous RAG scheduling* employed by MNEMORAG.

## IV. Design and Implementation

In this section, we present MNEMORAG, a correlation-aware RAG serving system that improves end-to-end efficiency by exploiting inter-request correlations and workload-level locality. Unlike conventional RAG systems that treat requests as independent execution units, MNEMORAG leverages cross-request redundancy to jointly optimize retrieval, prefix caching, and execution scheduling. The retrieval component is implemented with approximately 2.5K lines of code on top of Faiss (v1.8.0) [35] for scalable vector search, while the generation component adds around 0.5K lines of code on top of vLLM (v0.11.2) [36] for efficient LLM serving.

At a high level, MNEMORAG follows three design principles. (1) It exploits semantic correlations across requests to eliminate redundant similarity computation during retrieval. (2) It leverages document access locality to maximize prefix sharing and improve KV cache reuse during generation. (3) It decouples retrieval and generation across requests to enable cross-stage concurrency and improve GPU utilization.

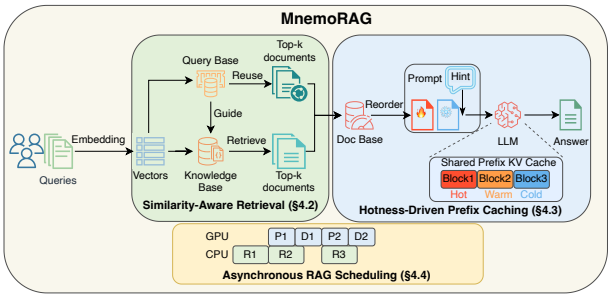



Fig. 6. MNEMORAG Architecture.

### A. MNEMORAG Overview

The overall architecture of MNEMORAG is illustrated in Figure 6. Given an input query, MNEMORAG executes a correlation-aware serving workflow that jointly optimizes retrieval, generation, and cross-request scheduling.

First, MNEMORAG performs *similarity-aware retrieval* by leveraging historical query–retrieval correlations maintained in a *Query Base*. Based on the similarity with previous queries, the system either directly reuses existing retrieval results or narrows the vector search space to relevant candidate clusters, thereby reducing redundant similarity computation.

Second, before LLM prefill, MNEMORAG applies *hotness-driven prefix caching* by exploiting document access locality maintained in a *Doc Base*. The system reorders retrieved documents according to their popularity to maximize shared prompt prefixes across requests. This reordering improves prefix KV cache reuse, while a lightweight *hint* preserves the original document ordering semantics for generation.

Finally, MNEMORAG employs *asynchronous RAG scheduling* to overlap retrieval and generation across requests. To

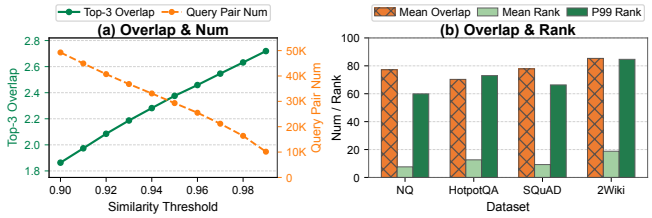



Fig. 7. Semantic similarity–induced overlap in retrieval. (a) Top-$k$ overlap versus similarity. (b) Probe overlap and rank statistics.

avoid contention between GPU-based retrieval and compute-intensive prefill, retrieval is scheduled after the current prefill phase completes, enabling effective cross-stage concurrency. This design reduces pipeline stalls, improves resource utilization, and enhances end-to-end serving throughput.

### B. Similarity-Aware Retrieval

Vector search in RAG retrieval typically follows a two-stage ANN pipeline. It first identifies the top-$nprobe$ nearest clusters, forming an ordered probe set $C = \{c_1, c_2, \ldots, c_{nprobe}\}$, and then performs fine-grained search within these clusters to retrieve the top-$k$ documents. Although this hierarchical design avoids exhaustive search over the entire corpus, probing multiple clusters remains a significant retrieval overhead.

Prior similarity-based systems, including GPTCache [66], Proximity [28], and QVCache [29], primarily use query similarity to determine cache reuse, thereby avoiding redundant retrieval or generation on cache hits. However, semantic similarity provides richer optimization opportunities beyond cache hits. MNEMORAG introduces *similarity-aware retrieval*, which leverages inter-request similarity as a continuous control signal to dynamically adjust retrieval computation, including probe budgets and candidate cluster selection.

This design exploits the property that semantic similarity induces convergence in retrieval behavior. Given two query vectors $v$ and $v'$ with cosine similarity $d(v, v') \geq \delta$, their retrieved documents and probed clusters exhibit substantial overlap. We quantify this convergence using two metrics: top-$k$ overlap $|R_k(v) \cap R_k(v')|$ and probe overlap $|C(v) \cap C(v')|$.

Empirical results (Figure 7) show that both metrics increase monotonically with inter-request similarity, indicating increasingly aligned retrieval behavior. As similarity increases, queries converge toward similar top-$k$ results, while their ANN searches increasingly focus on shared clusters rather than exploring the full index. For example, on the NQ [59] dataset with $k = 3$ and $nprobe = 128$, the top-3 overlap approaches 2.37 when similarity exceeds 0.95, and similar queries share 77.19 out of 128 probed clusters on average. Moreover, clusters containing top-$k$ results consistently appear among the earliest probes. These results reveal that semantically similar requests can share retrieval computation, enabling similarity-guided reduction of search cost.

Leveraging this retrieval convergence, MNEMORAG realizes *similarity-aware retrieval* as described in Algorithm 1. A lightweight, capacity-bounded *Query Base* stores historical queries and their corresponding top-$k$ results $R_k(v)$, capturing reusable retrieval signals without replicating the knowledge corpus. The *Query Base* employs LRU replacement, updating entry recency upon result reuse or similarity-guided retrieval. Queries served through direct reuse are not reinserted, while admitting a new query–result pair at capacity $B$ evicts the least recently used entry. With $B = 80$K, the *Query Base* occupies approximately 150 MB, bounding both storage and similarity-matching overhead regardless of cumulative request volume. Given a new query $v'$, the system identifies the most similar historical query $v$ with the highest cosine similarity using efficient GPU-based matrix operations and obtains the corresponding similarity score $s$ in approximately 0.37 ms on average (Lines 3–4). Two similarity thresholds, $\tau_1$ and $\tau_2$, then determine the retrieval strategy according to $s$:

**Top-$k$ Result Reuse (Lines 5–6).** If $s \geq \tau_1$, the high similarity between queries justifies directly reusing $R_k(v)$, thereby eliminating redundant probing. Based on empirical analysis, we set $\tau_1 = 0.95$ by default, reflecting that similarity scores above this threshold indicate sufficiently strong alignment between queries.

**Adaptive Retrieval (Lines 7–13).** If $\tau_2 \leq s < \tau_1$, retrieval is restricted to a similarity-adaptive probe space that balances efficiency and accuracy. We set $\tau_2 = 0.7$ by default to effectively leverage historical queries to guide the retrieval of new queries and maximize workload-level optimization. The probe budget is computed as

$$real_nprobe(s) = n_{\min} + (n_{\max} - n_{\min})(1 - \tilde{s})^{\gamma}, \quad (1)$$

where

$$\tilde{s} = \frac{s - s_{\min}}{s_{\max} - s_{\min}} \quad (2)$$

is the normalized similarity based on the maximum and minimum similarity values maintained via the *Query Base*. Here, $n_{\min}$ and $n_{\max}$ denote the minimum and maximum budgets on the number of clusters to probe, and $\gamma$ controls the decay rate of the function. The pruned cluster set is then defined as

$$C_{\text{pruned}} = \{c_1, \ldots, c_{real_nprobe(s)}\}, \quad (3)$$

and augmented with high-quality clusters derived from the top-$k$ results of the most similar query, denoted as $H(v)$:

$$P(v') = C_{\text{pruned}} \cup H(v), \quad (4)$$

ensuring retrieval is concentrated on semantically relevant regions while preserving coverage of high-quality candidate clusters. Typical parameter settings are $n_{\min} = 16$ and $n_{\max} = nprobe/2$, which empirically limit redundant cluster scans for highly similar queries while maintaining a sufficiently large probe budget to prevent escalation in accuracy. The decay factor $\gamma$ is set to 2, leveraging a convex reduction to accelerate the decrease of the probe budget for high-similarity queries, thereby concentrating the search on a smaller, more relevant subset of clusters.

**Original Retrieval (Lines 14–15).** When the similarity score falls below the lower threshold $\tau_2$, guidance from historical queries is considered unreliable, and the system

**Algorithm 1** Similarity-Aware Retrieval

**Variables:** new query $v'$; *Query Base* storing historical queries and their top-$k$ results; similarity thresholds $\tau_1, \tau_2$; probe budget bounds $n_{\min}, n_{\max}$; decay factor $\gamma$
**Output:** top-$k$ results $R_k(v')$
$(v, R_k(v)) \leftarrow \text{NEAREST_QUERY}(v', \text{Query Base})$
$s \leftarrow \text{SIMILARITY}(v, v')$
**if** $s \geq \tau_1$ **then** /* Result reuse */
    $R_k(v') \leftarrow R_k(v)$
**else if** $s \geq \tau_2$ **then** /* Adaptive retrieval */
    $\tilde{s} \leftarrow \text{NORMALIZE}(s)$
    $real_nprobe \leftarrow n_{\min} + (n_{\max} - n_{\min})(1 - \tilde{s})^{\gamma}$
    $C_{\text{pruned}} \leftarrow \text{TOP_CLUSTERS}(v', real_nprobe)$
    $H(v) \leftarrow \text{CLUSTERS_FROM_RESULTS}(R_k(v))$
    $P \leftarrow C_{\text{pruned}} \cup H(v)$
    $R_k(v') \leftarrow \text{TOPK_SEARCH}(v', P)$
**else** /* Original retrieval */
    $R_k(v') \leftarrow \text{TOPK_SEARCH}(v', \text{FULL_INDEX})$
**end if**
$\text{UPDATE_QUERYBASE}(v', R_k(v'))$
**return** $R_k(v')$

defaults to full-index probing to guarantee robust retrieval. In practice, as MNEMORAG operates over time, the *Query Base* accumulates historical queries, rendering this fallback increasingly infrequent.

### C. Hotness-Driven Prefix Caching

MNEMORAG introduces *hotness-driven prefix caching*, a mechanism that exploits skewed document access patterns to reduce prefill overhead during generation. The key idea is to treat frequently retrieved documents as stable computational anchors across requests, and to organize prompt construction and KV cache management accordingly. To this end, MNEMORAG maintains a lightweight *Doc Base* that tracks a *hotness score* for accessed documents, using raw access frequency as a proxy for relative importance. This abstraction enables the system to prioritize hot documents in both prompt structure and KV cache retention, thereby aligning KV cache management with the intrinsic workload distribution, while incurring negligible overhead.

To realize this design, MNEMORAG incorporates document hotness into prompt construction by querying the hotness scores of retrieved documents at an early stage of the generation process. As illustrated in Figure 8, based on their relative hotness, the system reorganizes these documents within the augmented prompt, placing frequently accessed ones earlier. Since prefix matching operates at the token level, such ordering increases the likelihood of cross-request prefix alignment, thereby improving KV cache reuse. However, this reordering may deviate from the original retrieval ranking and introduce the *Lost-in-the-Middle* effect [44], where content in the middle of the prompt receives disproportionately less attention. To address this, a lightweight *hint* encoding the original retrieval order is appended to the prompt. This design preserves the

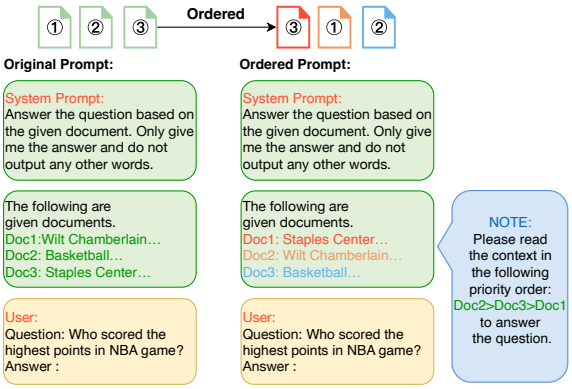



Fig. 8. Hotness-driven prompt construction.

semantic fidelity and interpretability of the retrieved context while retaining the efficiency gains of hotness-driven prefix reordering.

While hotness-aware prompt construction maximizes cross-request prefix alignment, sustaining this benefit under limited GPU memory requires coordinated KV cache placement. To this end, MNEMORAG organizes prefix KV blocks in a hierarchical GPU–CPU cache. The GPU tier retains latency-critical prefixes, whereas the CPU tier serves as a higher-capacity backing store for reusable blocks displaced from GPU memory. This hierarchy extends prefix reuse beyond the capacity of the GPU cache while avoiding the prefill cost of recomputing previously materialized prefixes.

To govern GPU residency, MNEMORAG employs a *hotness-aware eviction policy* that accounts for both document hotness and recency. Prefix KV blocks that are frequently and recently reused are preferentially retained on the GPU, whereas stale or infrequently accessed blocks are selected for demotion. Formally, we operationalize this policy via a priority-based eviction scheme, where the priority of a KV cache block $i$ is defined as

$$\text{Priority}_i = \frac{H_i}{\log(1 + \Delta t_i)} \tag{5}$$

where $H_i$ denotes the hotness score of the block's corresponding documents, reflecting its frequency of reuse, and $\Delta t_i$ represents the elapsed time since its last access. The logarithmic scaling balances the influence of hotness and recency, gradually reducing the priority of stale blocks. To enforce this policy, MNEMORAG maintains priority-aware metadata for each KV cache block, enabling the selective eviction of low-priority blocks under GPU memory pressure.

Under GPU memory pressure, low-priority blocks are demoted to CPU memory rather than discarded. Upon a subsequent prefix match, these host-resident blocks are restored to the GPU before prefill, while recomputation is required only for blocks absent from both tiers. This placement policy establishes a hot–warm–cold hierarchy: frequently reused blocks remain GPU-resident, less active but reusable blocks are retained in CPU memory, and stale blocks are removed when host capacity is exhausted. Although a CPU-cache hit incurs

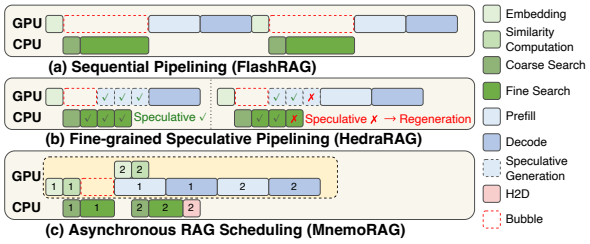



Fig. 9. Execution pipelines of conventional RAG workflows and MNEMORAG.

host-to-device transfer, it avoids redundant prefix computation and extends the effective KV cache capacity beyond GPU memory.

By combining KV cache management with hotness-aware prompt construction, MNEMORAG maximizes cross-request prefix reuse while adhering to GPU memory constraints. This holistic design improves prefix KV cache hit rate, reduces redundant prefill computation, and preserves the efficiency and fidelity of retrieval-augmented generation.

### D. Asynchronous RAG Scheduling

Despite extensive optimization of retrieval and generation in isolation, conventional RAG serving remains governed by stage-synchronous execution. Although retrieval must precede generation within each request, existing systems often extend this necessary dependency across requests, delaying subsequent retrieval until the current generation stage completes. This coarse-grained serialization creates pipeline bubbles: CPU retrieval resources remain idle during generation, while the GPU may subsequently stall awaiting the next generation-ready request.

Frameworks such as LangChain [16] and FlashRAG [17] preserve stage-level barriers between retrieval and generation, as illustrated in Figure 9(a). HedraRAG relaxes this barrier through fine-grained speculative pipelining, initiating generation before retrieval converges [34]. However, retrieval that outlasts the speculative window remains exposed on the critical path, while inaccurate speculation may necessitate correction or regeneration (Figure 9(b)). Consequently, speculative overlap does not guarantee that subsequent requests become generation-ready before the current generation completes.

To eliminate this unnecessary inter-request serialization, MNEMORAG introduces *asynchronous RAG scheduling* to reorganize the pipeline as illustrated in Figure 9(c). The system preserves the necessary intra-request dependency between retrieval and generation while decoupling their execution across requests. Retrieval remains the primary execution path and proceeds sequentially across admitted requests, whereas generation is submitted to vLLM `AsyncLLM` engine through an asynchronous coroutine registered with `asyncio.create_task`. Consequently, once retrieval for the current request completes, the scheduler immediately advances to the next request without awaiting the completion of ongoing generation. Within retrieval, query encoding is performed on the GPU, after which GPU-accelerated historical-query matching proceeds concurrently with CPU-based coarse cluster search; their results subsequently guide fine-grained vector search. Once the retrieved context and reusable prefix are determined, CPU-resident KV blocks are restored from pinned host memory through a dedicated H2D CUDA stream. Transfer completion is tracked with CUDA events, and prefill is admitted only after the required KV state becomes available. Combined with *similarity-aware retrieval*, which shortens the preparation path, this cross-request parallelism enables retrieval and KV restoration to be more effectively concealed by ongoing generation, thereby reducing pipeline stalls and sustaining more continuous GPU utilization.

## V. EVALUATION

**System.** We evaluate MNEMORAG on a CPU–GPU hybrid server. Vector search is executed using 64 physical cores from an AMD EPYC 7742 64-Core Processors, supported by 512 GB of host memory for the vector index and offloaded KV caches. LLM generation is performed on eight NVIDIA A100 40 GB NVLink GPUs.

**Models.** Our primary model is LLaMA3-8B [67]. To evaluate generality across model architectures and scales, we additionally include Qwen3-32B [68] and Llama3-70B [67], covering diverse LLM backbones.

**Corpus and Index.** We use Wikipedia [69] passages as the retrieval corpus (approximately 21M passages). Each passage is encoded into a 1024-dimensional dense vector using the E5-large embedding model [39]. For efficient large-scale retrieval, we construct an IVF4096 index, a widely adopted approximate nearest neighbor structure [53].

**Datasets.** We evaluate MNEMORAG on four widely used open-domain QA datasets: SQuAD (10,570 queries) [70], Natural Questions (79,168) [59], HotpotQA (90,447) [49], and 2WikiMultiHopQA (15,000) [71]. These datasets cover passage comprehension, single-hop factoid QA workloads and multi-hop reasoning workloads requiring bridging evidence, providing diverse evaluation scenarios.

**RAG Workflows.** We consider three representative RAG workflows: One-Pass [9], RECOMP [46], and iterative retrieval–generation (IRG) [51], corresponding to standard retrieval-then-generation, compression-augmented, and iterative retrieval–generation paradigms, respectively.

**Baselines.** We compare MNEMORAG with two representative RAG baselines: HedraRAG [34], a closely related state-of-the-art RAG system and FlashRAG [17], a modular open-source RAG framework. Together, they cover both standard configurable RAG pipelines and high-performance RAG serving, enabling a more comprehensive evaluation of MNEMORAG.

### A. Overall Improvement

We evaluate MNEMORAG through end-to-end experiments on four representative QA datasets, comparing it against three widely used RAG workflows under realistic serving conditions. Our evaluation focuses on overall system latency. We further analyze the impact of key retrieval parameters, including top-$k$ and $nprobe$, and examine the scalability of

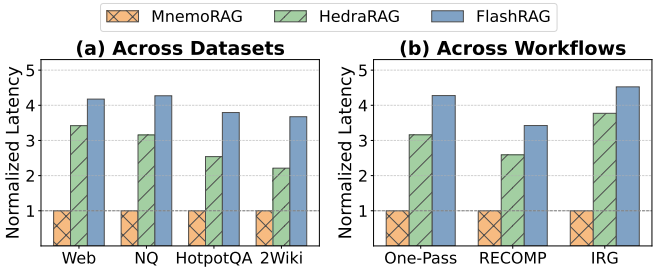



Fig. 10. End-to-end latency in RAG systems. (a) Across four datasets. (b) Across different RAG workflows on NQ dataset.

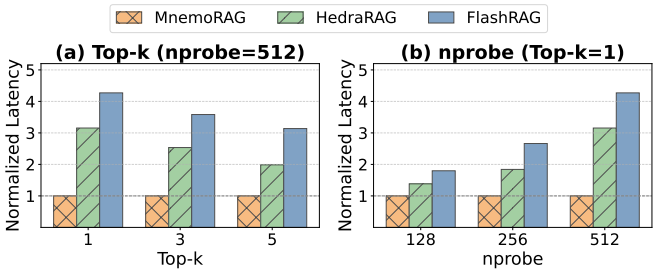



Fig. 11. Latency under different retrieval configurations in RAG systems. (a) Varying top-$k$ with $nprobe$ fixed at 512. (b) Varying $nprobe$ with top-$k$ fixed at 1.

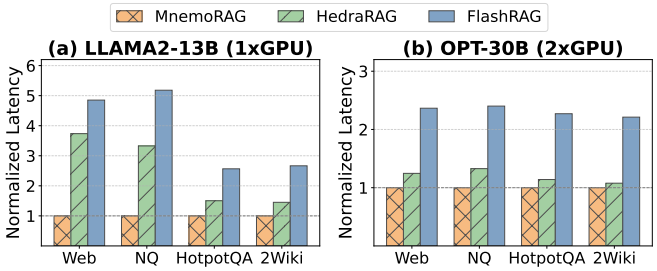



Fig. 12. Latency of RAG systems across datasets under different models. (a) LLaMA2-13B (Single-GPU). (b) OPT-30B (Multi-GPU).

MNEMORAG across larger LLMs and multi-GPU configurations. Unless otherwise specified, we set the request arrival rate to 24 req/s, with $nprobe = 512$, top-$k = 1$, and a retrieval batch size (RBS) of 256 for retrieval.

**End-to-End Performance Evaluation.** We first evaluate how MNEMORAG improves end-to-end latency compared to four QA datasets and representative RAG workflows. As illustrated in Figure 10, MNEMORAG consistently achieves substantial reductions in end-to-end latency in both datasets and RAG workflows. The normalized latency is reduced to $2.21\times$–$4.27\times$ across datasets and $2.59\times$–$4.52\times$ across workflows, with the largest improvements observed on NQ dataset and One-Pass workflow, respectively. These improvements persist as workflow complexity increases, yielding latency reductions of $3.42\times$ on RECOMP and $4.52\times$ on IRG, highlighting the effectiveness of MNEMORAG in mitigating compounded overhead in multi-stage retrieval–generation pipelines. This consistent advantage across both data and execution dimensions indicates that MNEMORAG effectively reduces the dominant sources of end-to-end latency, rather than optimizing for a specific workload or pipeline configuration. Even on optimized workflows such as IRG—where retrieval overhead is partially mitigated, MNEMORAG continues to deliver substantial gains, underscoring its complementary benefits to existing system-level optimizations.

In general, these results demonstrate that MNEMORAG provides systematic improvements in end-to-end latency, by eliminating redundant retrieval through cross-request similarity, improving prefix reuse via hotness-driven KV cache management, and overlapping retrieval with generation, effectively targeting the dominant bottlenecks in RAG serving.

**Performance Across Retrieval Parameters.** We further analyze the impact of key retrieval parameters on end-to-end latency, focusing on top-$k$ and $nprobe$. As shown in Figure 11(a), the relative latency gap between MNEMORAG and existing RAG frameworks gradually decreases as top-$k$ increases, with the normalized latency reducing from roughly $4.27\times$ to $1.98\times$. This trend arises because larger top-$k$ introduces more retrieved documents, which lengthens the generation phase and shifts the end-to-end latency composition toward generation. In addition, retrieving more documents reduces the prefix KV cache reuse opportunities, further weakening the relative advantage of latency reduction. Figure 11(b) illustrates the impact of varying $nprobe$, which controls the search scope of the vector index. As $nprobe$ increases, the normalized latency increases from approximately $1.38\times$ to $4.27\times$, reflecting the rapidly increasing retrieval cost caused by the expanded search space. Together, MNEMORAG consistently achieves lower end-to-end latency as retrieval complexity increases.

Collectively, these results show that MNEMORAG remains effective across different retrieval parameter configurations. This robustness stems from leveraging cross-request semantic similarity to eliminate redundant retrieval computation and mitigate the latency overhead introduced by larger search spaces.

**Performance Across LLM Backbones.** We also evaluate the scalability of MNEMORAG across model scales, including LLaMA2-13B and OPT-30B, in both single-GPU and multi-GPU settings. As shown in Figures 12, MNEMORAG achieves at least $1.45\times$ latency reduction on LLaMA2-13B and $1.07\times$ on OPT-30B compared to existing frameworks. Scaling to larger models shifts the bottleneck from retrieval to generation, introducing longer generation latency, higher memory pressure, and increased inter-GPU synchronization overhead. Despite this shift, MNEMORAG maintains a clear performance advantage, as baseline systems lack effective prefix KV cache reuse, incurring substantial redundant prefill computation under generation-dominated workloads. This trend indicates that the performance gains of MNEMORAG largely stem from effective prefix reuse rather than retrieval-specific optimizations alone. As bottlenecks concentrate on LLM inference, optimizing retrieval alone becomes insufficient, and overall efficiency is increasingly determined by the ability to minimize redundant prefill computation. By maximizing cross-request prefix reuse through hotness-driven caching, MNEMORAG improves KV cache hit rates and reduces prefill overhead, thereby maintaining robust performance across model scales and execution environments.

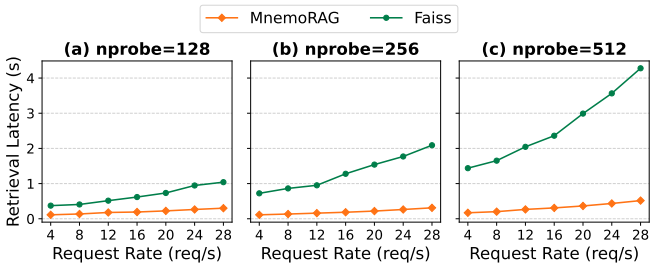



Fig. 13. Retrieval latency across increasing request arrival rates under varying $nprobe$ configurations.

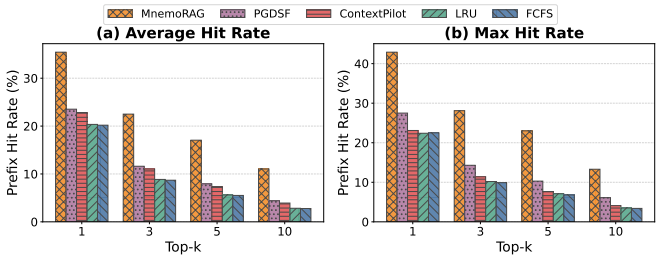



Fig. 14. Prefix KV cache hit rate under different top-$k$ settings.

## B. Performance Analysis

In this section, we conduct a comprehensive analysis of the three core optimization techniques in MNEMORAG , aiming to evaluate their individual contributions and their impact on end-to-end system performance.

**Retrieval Efficiency.** We systematically evaluate retrieval efficiency across a range of request arrival rates (4–28 req/s) and $nprobe$ configurations, capturing the retrieval behavior in realistic RAG serving scenarios. Figure 13 demonstrates that MNEMORAG consistently reduces retrieval latency compared to Faiss, achieving 2.87×–8.44× speedup (5.98× on average). The largest gain occurs at $nprobe = 512$, where the baseline incurs substantial overhead from exhaustive probing. Even under lighter configurations ($nprobe = 128$), MNEMORAG maintains a non-trivial advantage, indicating robustness across retrieval complexities. These gains stem from eliminating redundant index traversals and amortizing retrieval cost across requests, thus weakening the coupling between latency, concurrency, and probing depth. Consequently, MNEMORAG sustains a substantially flatter latency curve and better scalability under increasing load.

**Prefix KV Cache Hit Rate.** We evaluate prefix KV cache efficiency across varying top-$k$ settings to characterize prefix reuse under different retrieval granularities. Figure 14 shows that MNEMORAG consistently outperforms baselines employing LRU and FCFS policies, achieving up to 26.6% maximum prefix hit rate with top-$k = 1$, while improving the average prefix hit rate by 1.25×–1.82× and the maximum prefix hit rate by up to 2.16×. The performance gap widens as $k$ increases, where reduced prefix overlap limits the effectiveness of recency- and order-based policies. In contrast, MNEMORAG sustains substantial gains, indicating stronger robustness in low-redundancy regimes. The larger improvements in maximum prefix hit rate further suggest that MNEMORAG raises the upper bound of achievable prefix reuse. These gains arise from capturing structured prefix locality beyond simple temporal heuristics, enabling more effective prefix reuse across requests.

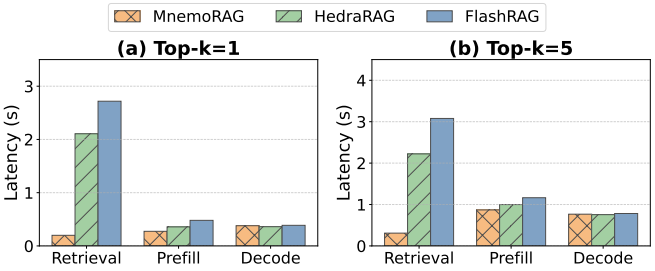



Fig. 15. Latency breakdown of retrieval, prefill, and decode across different top-$k$ settings.

**Retrieval–Generation Overlap.** We analyze the latency decomposition across retrieval, prefill, and decode under varying top-$k$ settings to examine the effectiveness of retrieval–generation overlap. Conventional RAG systems follow a sequential execution paradigm in which retrieval remains explicitly exposed on the critical path, significantly contributing to end-to-end latency and increasing with $k$. In contrast, Figure 15 shows that MNEMORAG substantially reduces the retrieval latency to a marginal level (around 0.20–0.31 s), making it consistently subordinate to the generation phase (prefill and decode). This relative latency relationship holds across top-$k$ settings and becomes more pronounced as $k$ increases due to the growth of generation latency. At top-$k = 1$, retrieval is already smaller than prefill (0.20 s vs. 0.27 s), and becomes insignificant compared to total generation at top-$k = 5$ (0.31 s vs. 1.64 s). Under this condition, retrieval can be fully overlapped with generation, thereby removing it from the critical path and eliminating its contribution to end-to-end latency. As a result, although overall latency still increases with $k$, this growth is driven solely by the scaling of the generation phase rather than the retrieval cost. Collectively, these results show that the gains of MNEMORAG arise from enabling complete retrieval–generation overlap, which depends on reducing the retrieval stage latency to remove it from the critical path.

# VI. Ablation Studies

In this section, we provide a fine-grained analysis of MNEMORAG through sensitivity studies and component-level ablations. We first examine how the similarity thresholds affect the latency-quality trade-off of similarity-aware retrieval. We then evaluate document ordering strategies to assess the impact of hotness-based reordering and the lightweight ordering hint on generation quality. Finally, we isolate the contribution of similarity-aware retrieval, hotness-driven prefix caching, and asynchronous scheduling. Unless otherwise specified, all experiments follow the default setup used in the main evaluation.

**Similarity Thresholds and Path Ablation.** We empirically determine the two similarity thresholds, where $\tau_1$ determines when top-$k$ Result Reuse is applied and $\tau_2$ determines when Adaptive Retrieval is enabled. With $\tau_2 = 0.85$, increasing $\tau_1$ beyond 0.95 improves Recall@3 by only 0.005 while

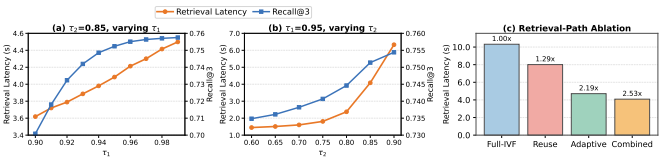



Fig. 16. Similarity-threshold sensitivity and retrieval-path ablation.

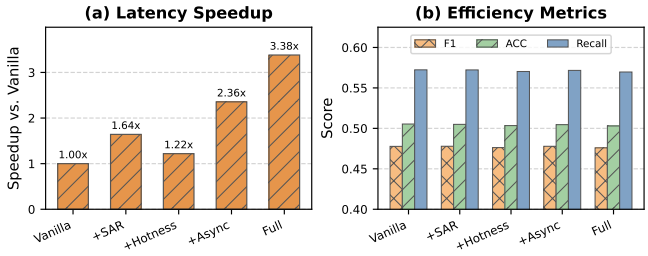



Fig. 17. Component-level ablation of MNEMORAG.

increasing retrieval latency from 4.08s to 4.50s; lower values progressively sacrifice recall. With $\tau_1 = 0.95$, increasing $\tau_2$ from 0.85 to 0.90 yields only a 0.003 recall gain but raises latency from 4.08s to 6.33s. These results identify $(\tau_1, \tau_2) = (0.95, 0.85)$ as the operating point that best balances retrieval quality and latency, achieving 0.751 Recall@3 and a 2.53× speedup over Full-IVF. The path ablation further shows 1.29× and 2.19× speedups from Result Reuse and Adaptive Retrieval alone. In practice, the thresholds can be calibrated on a small validation workload to select the lowest-latency configuration that satisfies a target retrieval-quality bound.

**Effect of Document Ordering.** We study the effect of document ordering by comparing three strategies: original retrieval order, hotness-based reordering, and hotness-based reordering with a lightweight ordering hint. Table I shows that the influence of reordering varies with the retrieval depth. For top-$k = 3$, the three strategies yield comparable quality, indicating limited sensitivity to document order under short contexts. For top-$k = 5$, Hotness+Hint achieves the best overall performance, improving F1/Acc/Recall by 4.1%/2.2%/0.9% over Hotness, suggesting that the hint helps preserve retrieval-order information while maintaining the benefits of hotness-based reordering. For top-$k = 10$, both reordered variants underperform the original order, although the hint partially recovers recall. This indicates that longer contexts introduce additional noise beyond ordering effects, limiting the effectiveness of lightweight order hints.

**Component Ablation.** We conduct a component-level ablation to quantify the contribution of each optimization in MNEMORAG. We use Vanilla RAG as the reference implementation, which is built directly with Faiss for retrieval and vLLM for generation, without framework-level orchestration or MNEMORAG-specific optimizations. As shown in Figure 17, similarity-aware retrieval (SAR) provides the largest latency reduction, achieving a 4.10× speedup over Vanilla RAG, while hotness-driven prefix caching and asynchronous scheduling provide complementary gains of 1.23× and 1.51×, respectively. Combining all components yields a 4.68× end-to-end speedup. The quality metrics indicate that the small performance degradation is mainly introduced by SAR due to approximate retrieval, whereas hotness-based reordering and asynchronous scheduling remain nearly lossless. This confirms that the three components play distinct roles: SAR reduces retrieval cost, hotness improves prefix reuse, and asynchronous scheduling overlaps CPU/GPU execution without materially affecting generation quality.

TABLE I
EFFECT OF DOCUMENT ORDERING STRATEGIES.

| Top-$k$ | Metric | Original | Hotness | Hotness+Hint |
|---|---|---|---|---|
| 3 | F1 | 0.4900 | 0.4877 | **0.4908** |
| | Acc | **0.5261** | 0.5174 | 0.5174 |
| | Recall | **0.5804** | 0.5736 | 0.5757 |
| 5 | F1 | 0.4485 | 0.4566 | **0.4752** |
| | Acc | 0.5268 | 0.5221 | **0.5338** |
| | Recall | 0.5779 | 0.5866 | **0.5917** |
| 10 | F1 | **0.3363** | 0.3167 | 0.3187 |
| | Acc | **0.5425** | 0.5018 | 0.5043 |
| | Recall | **0.5964** | 0.5588 | 0.5750 |

## VII. RELATED WORK

**Retrieval-Augmented Generation (RAG).** Recent research on RAG has explored both algorithmic and system-level optimizations to enhance efficiency and generation quality. At the algorithmic level, METIS [72] dynamically adapts RAG configurations on a per-query basis to balance latency and output fidelity. At the system level, prior works primarily target stage-specific optimizations. RAGCache [22] reduces redundant computation through knowledge-aware caching, enabling cross-query reuse of KV caches, but does not explicitly optimize the organization of retrieved documents to further improve cache reuse. PipeRAG [73] improves throughput for iterative retrieval–generation (IRG) workloads via pipelined execution enabled by algorithm–system co-design, while RAGO [33] provides a systematic framework for optimizing RAG serving through scheduling and resource management. In contrast, MNEMORAG adopts a holistic design that jointly optimizes retrieval and generation, addressing both redundancy and resource utilization to improve overall RAG serving efficiency.

**Retrieval Optimization.** Efficient retrieval remains a fundamental bottleneck in RAG systems and has been extensively studied in the context of large-scale vector search, particularly for IVF–based approximate nearest neighbor search (ANNS). SPANN [74] supports billion-scale vector search using a memory–disk hybrid architecture combining IVF partitioning with disk-resident posting lists. Auncel [75] further accelerates vector query execution by leveraging query-aware error–latency profiles to provide bounded accuracy and latency guarantees. Another line of research focuses on pruning distance computations: TriBase [76] enhances vector search efficiency through compression and reliable pruning guided by triangle inequalities. Differently, MNEMORAG improves retrieval efficiency in RAG workloads via *similarity-aware retrieval*, exploiting inter-request and inter-document correlations to eliminate redundant computations while preserving critical evidence.

**Generation Optimization.** A broad range of techniques have been proposed to improve LLM generation efficiency, with key–value (KV) caching emerging as a central mechanism. Existing work improves KV cache efficiency in two main directions. The first reduces the memory footprint via quantization [61], [64], [77]–[79], compression [80]–[84], and layer sharing [85]–[87], often at the cost of prediction accuracy. The second direction targets *prefix KV cache reuse* [36], [41]–[43], [61] to eliminate redundant computation across requests with shared prompt prefixes. However, most existing methods [36], [41], [43] rely on exact prefix matching, which limits cache hit rates, while approximate matching [42], [61] may degrade generation fidelity. MNEMORAG overcomes these limitations through *hotness-driven prefix caching*, strategically prioritizing frequently accessed documents to maximize cache utilization while preserving generation quality and robustness.

## VIII. CONCLUSION

We present MNEMORAG, a high-performance RAG system that achieves substantial efficiency gains by exploiting inter-request correlations. By integrating similarity-aware retrieval, hotness-driven prefix caching, and asynchronous RAG scheduling, MNEMORAG minimizes redundant vector computations, maximizes KV cache reuse, and alleviates cross-stage pipeline bottlenecks. Extensive evaluations across diverse RAG workflows and real-world datasets demonstrate that MNEMORAG consistently outperforms state-of-the-art systems, delivering significant speedups while maintaining retrieval accuracy and generation quality. These results underscore MNEMORAG's robustness, scalability, and practical effectiveness for high-performance RAG serving.

## REFERENCES


[1] A. H. Nasution and A. Onan, "Chatgpt label: Comparing the quality of human-generated and llm-generated annotations in low-resource language nlp tasks," *IEEE Access*, vol. 12, pp. 71 876–71 900, 2024.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[3] Y. Mo, H. Qin, Y. Dong, Z. Zhu, and Z. Li, "Large language model ai text generation detection based on transformer deep learning algorithm," 2024.

[4] S. Siriwardhana, R. Weerasekera, E. Wen, T. Kaluarachchi, R. Rana, and S. Nanayakkara, "Improving the domain adaptation of retrieval augmented generation models for open domain question answering," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1–17, 2023.

[5] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating llm hallucination via self reflection," in *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

[6] L. Zhou, W. Schellaert, F. Martínez-Plumed, Y. Moros-Daval, C. Ferri, and J. Hernández-Orallo, "Larger and more instructable language models become less reliable," *Nature*, vol. 634, no. 8032, 2024.

[7] B. Cao, D. Cai, L. Cui, X. Cheng, W. Bi, Y. Zou, and S. Shi, "Retrieval is accurate generation," *arXiv preprint arXiv:2402.17532*, 2024.

[8] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, and B. Qin, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, 2025.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, and T. Rocktäschel, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, 2020.

[10] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*, 2023, pp. 9802–9822.

[11] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-context retrieval-augmented language models," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023.

[12] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark *et al.*, "Improving language models by retrieving from trillions of tokens," in *International conference on machine learning*. PMLR, 2022, pp. 2206–2240.

[13] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, and T. Fu, "Siren's song in the ai ocean: A survey on hallucination in large language models," 2023.

[14] R. Shao, J. He, A. Asai, W. Shi, T. Dettmers, and S. Min, "Scaling retrieval-based language models with a trillion-token datastore," 2024.

[15] B. Wang, W. Ping, L. McAfee, P. Xu, B. Li, M. Shoeybi, and B. Catanzaro, "Instructretro: Instruction tuning post retrieval-augmented pretraining," 2023.

[16] *LangChain*, https://github.com/langchain-ai/langchain, 2026.

[17] J. Jin, Y. Zhu, X. Yang, C. Zhang, and Z. Dou, "Flashrag: A modular toolkit for efficient retrieval-augmented generation research," 2024.

[18] J. Zobel and A. Moffat, "Inverted files for text search engines," *ACM Computing Surveys*, vol. 38, no. 2, p. 6, 2006.

[19] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 754–17 762.

[20] M. Shen, M. Umar, K. Maeng, G. E. Suh, and U. Gupta, "Towards understanding systems trade-offs in retrieval-augmented generation model inference," 2024.

[21] Y. Zhu, J.-C. Gu, C. Sikora, H. Ko, and Y. Liu, "Accelerating inference of retrieval-augmented generation via sparse context selection," 2024.

[22] C. Jin, Z. Zhang, X. Jiang, F. Liu, S. Liu, X. Liu, and X. Jin, "Ragcache: Efficient knowledge caching for retrieval-augmented generation," *ACM Transactions on Computer Systems*, vol. 44, no. 1, pp. 1–27, 2025.

[23] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "{Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}," in *2024 USENIX annual technical conference (USENIX ATC 24)*, 2024, pp. 111–126.

[24] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "Realm: Retrieval-augmented language model pre-training," 2020.

[25] M. Lewis, M. Ghazvininejad, G. Ghosh, A. Aghajanyan, S. Wang, and L. Zettlemoyer, "Pre-training via paraphrasing," in *Advances in Neural Information Processing Systems*, 2020.

[26] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," 2024.

[27] P. Peng, W. Lin, W. Wu, X. Wang, and Y. Liu, "Has: Accelerating rag through homology-aware speculative retrieval," *arXiv preprint arXiv:2604.20452*, 2026.

[28] S. Bergman, A.-M. Kermarrec, D. Petrescu, R. Pires, M. Randl, M. De Vos, and J. Zhang, "Leveraging approximate caching for faster retrieval-augmented generation," in *Proceedings of the 26th International Middleware Conference*, 2025, pp. 340–353.

[29] A. E. Göçer, I. Tsakalidou, H. Nicholson, K. Kim, and A. Ailamaki, "Qvcache: A query-aware vector cache," *arXiv preprint arXiv:2602.02057*, 2026.

[30] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "Cacheblend: Fast large language model serving for rag with cached knowledge fusion," in *Proceedings of the twentieth European conference on computer systems*, 2025, pp. 94–109.

[31] S. Agarwal, S. Sundaresan, S. Mitra, D. Mahapatra, A. Gupta, R. Sharma, N. J. Kapu, T. Yu, and S. Saini, "Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–28, 2025.

[32] Y. Jiang, Y. Huang, L. Cheng, C. Deng, X. Sun, and L. Mai, "Contextpilot: Fast long-context inference via context reuse," *Proceedings of Machine Learning and Systems*, vol. 8, pp. 1639–1655, 2026.

[33] W. Jiang, S. Subramanian, C. Graves, G. Alonso, A. Yazdanbakhsh, and V. Dadu, "Rago: Systematic performance optimization for retrieval-

augmented generation serving," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 974–989.

[34] Z. Hu, V. Murthy, Z. Pan, W. Li, X. Fang, Y. Ding, and Y. Wang, "Hedrarag: Co-optimizing generation and retrieval for heterogeneous rag workflows," in *Proceedings of the ACM SIGOPS 31st symposium on operating systems principles*, 2025, pp. 623–638.

[35] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *IEEE Transactions on Big Data*, 2025.

[36] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.

[37] R. Dayarathne, U. Ranaweera, and U. Ganegoda, "Comparing the performance of llms in rag-based question-answering: A case study in computer science literature," in *International Conference on Artificial Intelligence in Education Technology*. Springer, 2024, pp. 387–403.

[38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.

[39] L. Wang, N. Yang, X. Huang, and B. Jiao, "Text embeddings by weakly-supervised contrastive pre-training," 2022.

[40] L. Wang, N. Yang, X. Huang, and L. Yang, "Multilingual e5 text embeddings: A technical report," 2024.

[41] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "Sglang: Efficient execution of structured language model programs," *Advances in neural information processing systems*, vol. 37, pp. 62 557–62 583, 2024.

[42] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt cache: Modular attention reuse for low-latency inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 325–338, 2024.

[43] L. Ye, Z. Tao, Y. Huang, and Y. Li, "Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 11 608–11 620.

[44] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.

[45] G. Kim, S. Kim, B. Jeon, J. Park, and J. Kang, "Tree of clarifications: Answering ambiguous questions with retrieval-augmented large language models," 2023.

[46] F. Xu, W. Shi, and E. Choi, "Recomp: Improving retrieval-augmented lms with context compression and selective augmentation," in *The Twelfth International Conference on Learning Representations*, 2023.

[47] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," 2024.

[48] Y. Yu, W. Ping, Z. Liu, B. Wang, and J. You, "Rankrag: Unifying context ranking with retrieval-augmented generation in llms," 2024.

[49] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2018, pp. 2369–2380.

[50] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, and Q. Liu, "Active retrieval augmented generation," 2023.

[51] Z. Shao, Y. Gong, Y. Shen, and M. Huang, "Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy," 2023.

[52] Z. Yue, H. Zhuang, and A. Bai, "Inference scaling for long-context retrieval augmented generation," 2024.

[53] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[54] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[55] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[56] A. Babenko and V. Lempitsky, "The inverted multi-index," *IEEE transactions on pattern analysis and machine intelligence*, vol. 37, no. 6, pp. 1247–1260, 2014.

[57] R. Yu, W. Huang, S. Bai, J. Zhou, and F. Wu, "Aquapipe: A quality-aware pipeline for knowledge retrieval and large language models," *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–26, 2025.

[58] M. Ahmed, R. Seraj, and S. M. S. Islam, "The k-means algorithm: A comprehensive survey and performance evaluation," *Electronics*, vol. 9, no. 8, 2020.

[59] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee *et al.*, "Natural questions: a benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 453–466, 2019.

[60] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.

[61] Y. Liu, H. Li, K. Du, J. Yao, Y. Cheng, Y. Huang, S. Lu, M. Maire, H. Hoffmann, A. Holtzman *et al.*, "Cachegen: Fast context loading for language model applications," *arXiv preprint arXiv:2310.07240*, 2023.

[62] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "Efficiently programming large language models using sglang." 2023.

[63] Y. Li, R. Gu, C. Huan, Z. Wang, R. Yao, C. Tian, and G. Chen, "Hotprefix: Hotness-aware kv cache scheduling for efficient prefix sharing in llm inference systems," *Proceedings of the ACM on Management of Data*, vol. 3, no. 4, pp. 1–27, 2025.

[64] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.

[65] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, 2022, pp. 521–538.

[66] F. Bang, "Gptcache: An open-source semantic cache for llm applications enabling faster answers and cost savings," in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, 2023, pp. 212–218.

[67] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[68] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[69] D. Chen, A. Fisch, J. Weston, and A. Bordes, "Reading wikipedia to answer open-domain questions," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 1870–1879.

[70] J. Berant, A. Chou, R. Frostig, and P. Liang, "Semantic parsing on freebase from question-answer pairs," in *Proceedings of the 2013 conference on empirical methods in natural language processing*, 2013, pp. 1533–1544.

[71] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps," in *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, pp. 6609–6625.

[72] S. Ray, R. Pan, Z. Gu, K. Du, S. Feng, G. Ananthanarayanan, R. Netravali, and J. Jiang, "Metis: fast quality-aware rag systems with configuration adaptation," in *Proceedings of the ACM SIGOPS 31st symposium on operating systems principles*, 2025, pp. 606–622.

[73] W. Jiang, S. Zhang, B. Han, J. Wang, B. Wang, and T. Kraska, "Piperag: Fast retrieval-augmented generation via algorithm-system co-design," *arXiv preprint arXiv:2403.05676*, 2024.

[74] Q. Chen, B. Zhao, H. Wang, M. Li, C. Liu, Z. Li, M. Yang, and J. Wang, "Spann: Highly-efficient billion-scale approximate nearest neighborhood search," *Advances in Neural Information Processing Systems*, vol. 34, pp. 5199–5212, 2021.

[75] Z. Zhang, C. Jin, L. Tang, X. Liu, and X. Jin, "Fast, approximate vector queries on very large unstructured datasets," in *20th USENIX Symposium*

*on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 995–1011.

[76] Q. Xu, J. Yang, F. Zhang, J. Pan, K. Chen, Y. Shen, A. C. Zhou, and X. Du, "Tribase: A vector data query engine for reliable and lossless pruning compression using triangle inequalities," *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–28, 2025.

[77] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao, "Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm," *arXiv preprint arXiv:2403.05527*, 2024.

[78] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," *arXiv preprint arXiv:2402.02750*, 2024.

[79] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, "Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 3304–3331, 2024.

[80] M. Adnan, A. Arunkumar, G. Jain, P. J. Nair, I. Soloveychik, and P. Kamath, "Keyformer: Kv cache reduction through key tokens selection for efficient generative inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 114–127, 2024.

[81] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, "Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time," *Advances in Neural Information Processing Systems*, vol. 36, pp. 52 342–52 364, 2023.

[82] D. Yang, X. Han, Y. Gao, Y. Hu, S. Zhang, and H. Zhao, "Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3258–3270.

[83] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.

[84] Z. Li, E. Wallace, S. Shen, K. Lin, K. Keutzer, D. Klein, and J. Gonzalez, "Train big, then compress: Rethinking model size for efficient training and inference of transformers," in *International Conference on machine learning*. PMLR, 2020, pp. 5958–5968.

[85] Y. Xiong, H. Wu, C. Shao, Z. Wang, R. Zhang, Y. Guo, J. Zhao, K. Zhang, and Z. Pan, "Layerkv: Optimizing large language model serving with layer-wise kv cache management," *arXiv preprint arXiv:2410.00428*, 2024.

[86] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 4895–4901.

[87] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.